Eventually Consistent Access Control with Deterministic Deny-Wins Replay for Multi-Stakeholder Offline Systems

Aditya Sissodiya, Johan Kristiansson, Eric Chiquito, Ulf Bodin
Luleå University of Technology, Sweden

Abstract—Offline maintenance, federated data sharing, and similar settings need authorization decisions that can be made locally and later reconciled after partitions heal. We present EQUACK, an eventually consistent access-control model that represents data updates, grants/revokes, and trust updates in a single signed causal event log. Replicas merge logs by set union and compute the same materialized state by deterministic replay of the causal DAG using a deterministic linearization and a deny-wins authorization gate. Authorization evidence is carried in-band using W3C Verifiable Credentials and time-versioned status-list updates, enabling offline verification without a central PDP. When confidentiality is required, payloads can be encrypted per tag and key-rotated; ciphertext remains replicated for convergence, while decryption and redaction occur at query time.
We evaluate EQUACK, a Rust prototype, using deterministic synthetic workloads and policy-heavy CBOR logs. In randomized delivery tests (100 topological permutations of a 10k-op log), all replicas converged to an identical state digest. Deny-wins revocation skipped all post-revocation writes (500/500), and multi-issuer conflicts resolved deterministically. Replay scales approximately linearly with log size; linear chains reach up to 45k ops/s in our implementation and checkpoints yield 8–11$\times$ speedup. We also demonstrate partition-merge invalidation of post-revocation operations and audit tamper detection relative to a pinned checkpoint digest. Encryption adds $\sim$6.5% overhead in small-scale tests.

Index Terms—Access control, revocation, verifiable credentials, CRDTs, deterministic replay, audit, offline systems, industrial IoT, data spaces.

1 INTRODUCTION

Access control in multi-stakeholder environments faces a fundamental tension; devices and services must continue operating when disconnected from central infrastructure, yet security-critical decisions—especially credential revocation—must eventually be enforced consistently across all parties. This tension is particularly acute in industrial IoT [1], [2], healthcare [3], and cross-organizational data sharing [4], [5], where equipment changes hands between manufacturers, operators, and service providers throughout its lifecycle.

A factory operates hundreds of programmable controllers on a restricted network. Maintenance is performed by engineers from different vendors who are granted temporary access to specific machines. Controllers may run for days without reliable connectivity [6], [7]. An engineer can update a safety parameter while offline using a credential issued by their employer; hours later, that credential may be revoked after a safety incident. When logs from different controllers are later synchronized, the system must deterministically agree on which parameter changes were authorized and produce an audit trail showing who changed what, under which permission, and when.

Similar tensions arise in federated research data sharing [3], [4] and disaster response coordination, where independent authorities, offline devices, and evolving permissions create the same enforcement gap. Across these settings, authorization is an *offline-first* problem: nodes must decide locally, yet the overall system must later reconcile those decisions into a consistent and auditable authorization state.

Kleppmann et al. [8] identify access control as a fundamental open problem for local-first software built on CRDTs, observing that enforcing access rights in a decentralized setting—particularly revoking access—remains unsolved. This results in the following problem statement.

How can multiple independent authorities issue and revoke fine-grained permissions for shared resources that operate offline for extended periods, while ensuring that all parties eventually converge on the same authorized state and can verify a tamper-evident history of access decisions?

We call the model EQUACK (Eventually Consistent Access Control). Our Rust prototype implementation is *equack-core*, which we use for all experiments.

This paper makes the following contributions:

1) *Problem analysis* (Section 2): Conceptual requirements derivation for offline multi-stakeholder authorization, gap analysis of four existing paradigms, and a mechanism-level comparison across five axes (Section 2.6).
2) EQUACK *model* (Section 3): An eventually consistent access-control model with formal guarantees for determinism, convergence, policy safety, audit integrity, and leaver security.
3) *Deny-wins replay semantics* (Section 3): Causal DAG ordering with deny-wins enforcement ensuring revoked credentials have no post-reconciliation effect.

4) *Implementation and evaluation* (Sections 4–5): The EQUACK Rust prototype with empirical validation across 14 experiments covering convergence, revocation, audit, performance, and multi-node sync.

2 Problem Analysis

The offline multi-stakeholder authorization problem arises when systems must (i) continue operating during network partitions, (ii) support multiple independent authorities that can grant and revoke permissions, and (iii) eventually converge to a consistent, auditable authorization state. This section defines the problem context, presents conceptual requirements derived from motivating use cases, and evaluates existing approaches against those requirements.

Before defining requirements or comparing paradigms, we state the assumptions that shape our problem space. Section 2.2 formalizes the full system and threat model; here we state only the assumptions needed to interpret R1–R6 and the subsequent comparisons.

- **Offline-first operation.** Replicas may be disconnected for extended periods (hours to days). Solutions requiring continuous connectivity to a central PDP or global consensus are excluded by construction.
- **Multi-authority governance.** Multiple independent organizations issue and revoke credentials. No single root authority controls all permissions.
- **Eventual delivery.** All well-formed operations are eventually delivered to all correct replicas, though delivery order is arbitrary and may be adversarially influenced.
- **Adversary capabilities.** An adversary may delay, reorder, or partition messages; compromise subject credentials; and attempt to tamper with local logs. The adversary is computationally bounded.
- **Out of scope.** We do not solve full Byzantine governance (colluding majorities of issuers) or physical key compromise. These require additional mechanisms.

This context explains why common fixes fail: online introspection endpoints violate offline-first; global consensus violates availability during partition; single-authority trust violates multi-stakeholder governance.

2.1 The Offline Enforcement Problem

Consider a set of replicas $R = \{r_1, \ldots, r_n\}$ operated by multiple stakeholders, maintaining shared resources under access control. Each replica may be offline for extended periods, during which it must make local authorization decisions. When replicas reconnect, they must reconcile their local states into a globally consistent outcome.

The core difficulty is not issuing permissions—standard PKI and token mechanisms suffice for that—but ensuring that all replicas eventually agree on *which operations were authorized at the time they occurred*, despite:

- **Partition-time divergence**: Different replicas may observe different subsets of grants, revokes, and data operations during a partition.
- **Out-of-order delivery**: A revocation may arrive at a replica after operations it should have invalidated.
- **Multi-authority conflicts**: Independent issuers may make conflicting decisions (e.g., one grants while another revokes) that must be resolved deterministically.

Jacob and Hartenstein [9] formalize this limitation by defining *Eventually Consistent Access Control* (ECAC) as a conceptual model in which system entities autonomously decide and enforce access control to the best of their knowledge and belief, replacing up-front coordination with subsequent reconciliation. Their ECAC model distinguishes causal authorization (immutable, based on predecessor history) from topological authorization (mutable, based on the total execution order), and provides formal safety and liveness properties—including app state confluence under non-monotonic revocations—proven on an abstract algorithm derived from Matrix's state resolution. However, Jacob and Hartenstein do not propose a concrete mechanism for credential-based authorization epochs, deny-wins replay semantics, or in-band trust evidence; their authorization model is based on Matrix's level- and attribute-based access control (LeABAC) rather than verifiable credentials with policy-engine evaluation. Furthermore, they provide no implementation or experimental evaluation. EQUACK addresses these gaps: it instantiates the ECAC paradigm with a concrete deny-wins replay mechanism over VC-based authorization epochs, Cedar policy evaluation, a tamper-evident audit trail, and an evaluated Rust prototype.

2.2 System Model

We consider a set of replicas operated by multiple stakeholders (organizations) that jointly maintain a replicated log of operations over shared resources. Replicas are deployed in environments with intermittent connectivity (e.g., edge and industrial deployments) and may be offline for extended periods. Each replica locally stores all received operations and materializes application state by deterministic replay.

A *subject* (principal) is an actor that initiates operations (users, services, devices). One or more independent *issuers* create and revoke time-bounded credentials for subjects. Each replica hosts a policy enforcement point (PEP) that admits subject requests only when locally authorized. The PEP then emits a signed log event that carries the subject identity and authorization evidence (e.g., the VC). Importantly, the event timestamp (HLC) and the event signature are produced by the replica/PEP, not by the subject, preventing a compromised subject from forging or backdating log entries under our threat model.

All state changes—data updates, grants, revokes, and trust updates—are represented as signed operations in a causal DAG. Replicas disseminate operations using gossip/anti-entropy and fetch missing dependencies (parents) until replay can proceed. We assume an asynchronous network: messages can be delayed, reordered, duplicated, or temporarily dropped.

2.3 Threat Model and Assumptions

EQUACK targets security failures that arise specifically from *offline enforcement* and *multi-authority governance*: stale credentials, conflicting grants and revokes across partitions, and post-hoc manipulation of authorization history.

We assume an adversary may (i) compromise subject credentials (e.g., steal a valid VC or authentication token) and attempt to request unauthorized operations, (ii) control the network to delay, reorder, or partition replicas, and (iii) attempt to tamper with stored history or audit records on one or more replicas. However, the adversary cannot forge replica/PEP signing keys or arbitrarily choose the HLC timestamps of accepted log events. The adversary is computationally bounded and cannot break standard cryptographic primitives.

The following assumptions scope the guarantees claimed by EQUACK:

1) **Honest-but-buggy administrators.** Issuers and policy administrators follow protocols but may make configuration errors; EQUACK provides safety-by-default semantics that limit the effect of ambiguous concurrent decisions.
2) **Eventual delivery.** Every well-formed operation is eventually delivered to every correct replica (e.g., via anti-entropy), although delivery order is arbitrary.
3) **No physical compromise of key material.** Long-term private keys and tag keys are not exfiltrated via physical attacks on devices; compromise is modeled at the software/account level.
4) **Bounded partition duration.** Offline periods are finite; after reconnection, replicas reconcile by exchanging missing operations.

EQUACK does not attempt to provide safety against a fully Byzantine ecosystem in which all participants collude, nor does it address side-channel leakage or cryptographic primitive failures. Metadata privacy (hiding access patterns or policy structure) is treated as future work.

2.4 Conceptual Requirements

From the system context and motivating use cases (Section 1), we derive six conceptual requirements for offline multi-stakeholder access control. These serve as criteria for comparing existing paradigms and guiding EQUACK's design.

1) **R1—Offline enforceability.** Authorization decisions must be enforceable without contacting a central PDP or ledger; disconnected nodes operate using locally verifiable evidence.
2) **R2—Deterministic post-partition convergence.** Given the same set of events, all parties must compute the same final authorization state regardless of message arrival order.
3) **R3—Revocation safety under partition.** Revocations must deterministically invalidate effects in the replicated state after reconciliation, even if the revocation arrives after the operations it should have blocked. Revocation applies to the digital state; irreversible physical-world actions require compensating controls at the application layer.
4) **R4—Tamper-evident accountability.** The history of grants, revokes, and authorization decisions must form an append-only, verifiable trace.
5) **R5—Decentralized multi-authority issuance.** Independent issuers must grant and revoke permissions without coordination; verifiers check authenticity offline using locally available evidence.
6) **R6—Confidentiality with post-revocation protection.** Unauthorized parties should not read protected content; after revocation-triggered key rotation, revoked principals cannot decrypt data created after their access ended.

We do not claim a general solution to Byzantine fault tolerance; EQUACK assumes honest-but-buggy administrators and does not address scenarios where a quorum of issuers collude maliciously.

2.5 Gap Analysis

Table 1 evaluates five dominant approaches against R1–R6. Centralized IAM couples enforcement to online coordination, failing R1 and R3 under partition [9]–[11]. Token-based schemes enable offline checks but lack deterministic reconciliation and timely revocation [12]–[15]. CRDTs provide data convergence but do not define authorization semantics or revocation rollback [16], [17]. Blockchains offer total ordering and auditability but require online consensus incompatible with offline enforcement [18]–[25]. Capability-based systems [26]–[28] enable offline verification of delegated authority but lack deterministic convergence of conflicting authorization state and retroactive revocation. No existing approach simultaneously satisfies R1–R6, motivating EQUACK's design.

2.6 Mechanism-Level Comparison

To clarify EQUACK's positioning, Table 2 compares it against the closest design patterns along five standard axes [29]–[36]: ordering model, revocation semantics, trust distribution, audit model, and offline availability.

EQUACK fills the gap between these approaches: CRDT replication provides data convergence but not authorization convergence [16], [17]; ledgers provide total order but require consensus; token/VC designs support offline checks but lack deterministic reconciliation [12]. EQUACK's causal DAG linearization plus deny-wins gate makes authorization a reproducible function of the reconciled event set without online consensus.

2.6.0.1 Capability-based authorization.: Capability-based authorization constitutes a fifth paradigm not captured by earlier versions of Tables 1 and 2. Systems such as Macaroons [26], UCAN [27], Biscuit [37], ZCAP-LD, and WAVE [28] enable offline verification of delegated authority via cryptographic token chains: a holder can attenuate and present capabilities without contacting the issuer. The Willow Protocol [38] couples peer-to-peer data synchronization with Meadowcap, a capability system supporting read/write authorization with delegation and attenuation. These designs satisfy R1 (offline verification) and R5 (multi-authority delegation) but do not address R2 (deterministic convergence of conflicting authorization state across partitions) or R3 (retroactive invalidation of effects authorized under stale evidence). Revocation in capability systems either relies on expiry, requiring short-lived tokens and frequent re-issuance, or on propagation of revocation lists, which faces the same freshness problem as VC status lists. EQUACK's contribution is orthogonal: it provides the

TABLE 1
Gap analysis against conceptual requirements R1–R6. "Partial" indicates the requirement can be achieved only with additional coordination or significant residual risk.

Requirement	Centralized IAM	Token-based	CRDT-only	Blockchain	Capability-based
R1 Offline enforceability	No	Partial	Yes	No	Yes
R2 Deterministic convergence	Yes	No	Yes	Yes	No
R3 Revocation safety	No	No	No	Partial	Partial
R4 Tamper-evident audit	No	No	No	Yes	Partial
R5 Multi-authority issuance	No	No	No	Yes	Yes
R6 Confidentiality	Partial	Partial	No	No	No

TABLE 2
Mechanism-level comparison of EQUACK to closest design alternatives. Each column represents a design paradigm; rows correspond to the five comparison axes.

Axis	CRDT-only replication	Token/VC offline checks	Ledger-based AC	Capability-based	EQUACK
Ordering model	Partial order / join-semilattice merge; no global history	No shared history; decisions are local + issuer freshness	Total order via consensus / finality	No shared history; delegation chains are acyclic but unordered	Causal DAG + deterministic linear extension (Alg. 1)
Revocation semantics	Not encoded into merge; revoked effects may persist post-merge	Depends on freshness/online status; partition window persists	Globally ordered but requires online consensus to commit	Expiry-based or propagation-dependent; no retroactive invalidation	Deny-wins epochs: REVOKE deterministically terminates prior/concurrent GRANT epochs; post-revoke effects skipped on reconciliation
Trust distribution	Out-of-band PKI; data convergence only	Out-of-band issuer infra (status endpoints, OCSP)	Trust anchored in ledger membership / consensus	Delegation chains from root capabilities; offline-verifiable	In-band TrustView: issuer keys + status evidence replicated as events; offline validation from pinned genesis
Audit model	Data history not necessarily reconstructible; depends on app-level logging	Depends on external logging; hard to reconstruct distributed decisions	Immutable ledger provides auditability (at consensus cost)	Token chain provides delegation provenance; no operation-level audit	Signed + hash-chained audit records; decisions reproducible from the log against pinned checkpoint digest
Offline availability	Yes for writes, but no safe authorization rollback	Partial (offline permit) but revocation timeliness requires online checks	No (must reach consensus to commit)	Yes for authorization checks; freshness-limited for revocation	Yes: local enforcement continues; safety repaired deterministically on merge

conflict-resolution and replay mechanism that deterministically reconciles divergent authorization decisions after partition healing—a problem that capability chains alone do not address.

2.6.0.2 Zanzibar-family and policy engines.: Zanzibar [39] and its derivatives (SpiceDB, OpenFGA [40]) provide relationship-based authorization backed by strongly consistent stores (Spanner in Zanzibar's case). The Zookie mechanism ensures that a check never returns stale results by encoding a consistency token tied to the latest write—an approach that requires always-on connectivity and globally consistent infrastructure, violating R1 by construction. Policy engines such as OPA and Cedar [41] support offline policy *evaluation* via pre-loaded policy bundles, but not offline policy *modification* with deterministic convergence: when two disconnected nodes independently modify authorization state (e.g., one grants while another revokes), these engines provide no reconciliation mechanism. EQUACK occupies the gap between these: it supports offline operation (like capability tokens) while providing deterministic post-partition convergence of authorization state (like centralized systems), without requiring global consensus.

2.6.0.3 Matrix State Resolution.: The closest deployed system solving a structurally similar problem is Matrix's state resolution algorithm (v2) [42]. Like EQUACK, Matrix maintains a DAG of events and deterministically resolves state across federated servers. However, Matrix's authorization model is power-level-based: authorization events are prioritized by event type and sender power level, with specific tie-breaking rules (lexicographic event ID comparison). EQUACK differs in using CRDT formalism for data convergence, deny-wins semantics for authoriza-

tion safety, and VC-based authorization epochs rather than power-level assignments. Jacob and Hartenstein [9] provide a formal analysis of Matrix's approach as an instance of ECAC; EQUACK builds on this conceptual foundation with a concrete mechanism targeting industrial multi-stakeholder settings.

3 EQUACK MODEL

EQUACK is an Eventually Consistent Access Control model designed for offline-first operation with rigorous security and auditability. This section presents the conceptual model and its formal guarantees.

EQUACK combines four key elements: (1) a causal event log with deterministic deny-wins replay, (2) verifiable credentials for time-bounded authorization epochs, (3) a hybrid ABAC/RBAC policy model with deny-overrides semantics, and (4) per-tag encryption with key rotation.

We outline formal guarantees for convergence, safety, determinism, audit integrity, and leaver security (post-revocation confidentiality under key rotation). The system's four-phase lifecycle (Ingest, Validate, Replay, Query) is summarized in Figure 1. The remainder of this section expands each phase and uses subsequent figures to clarify how the causal event log, in-band trust, deny-wins gating, and (optional) confidentiality interact to produce a deterministic authorization outcome.

3.1 Event Log and Deterministic Deny-Wins Replay

EQUACK maintains an append-only event log structured as a causal DAG, where each event points to its causal predecessors. The DAG encodes happens-before relations, and system state is derived by deterministically sorting and replaying events. A fixed traversal order ensures every replica produces the same sequence.

Concurrent events—those without causal order—are resolved using a composite key: a Hybrid Logical Clock (HLC) value plus a unique operation ID. The HLC combines physical and logical time to preserve causality; when timestamps match, op IDs break ties deterministically. Replay thus becomes a pure function of the event set, guaranteeing identical state across replicas (R2).

Separation of DAG and CRDT roles. A common point of confusion in offline access control is the overlapping role of causal graphs and Data CRDTs. In EQUACK, the pipeline follows a strict separation of concerns: the DAG and topological sort fix a deterministic, totally-ordered sequence of events to evaluate *authorization and policy validity*. Meanwhile, the CRDT state (e.g., MVReg, OR-Set) strictly handles *data payload convergence*. The policy engine evaluates the strict ordered DAG to build epochs and TrustViews; only authorized DATA events are then handed off to the CRDTs.

The event DAG acts as a convergent CRDT: merging logs is a set union—commutative, associative, idempotent. Operation structure and DAG mechanics are shown in Figure 2. Each operation carries a signed header with parent links, HLC timestamp, producer key, and typed payload. Replicas accept events opportunistically but only replay them once causal predecessors are present (activated/pending staging), ensuring every replica topologically sorts the same DAG prefix.

Replay follows a deny-wins rule: each event is applied only if the actor held the required permission at that moment; unauthorized events become no-ops. Operations are disseminated via gossip and anti-entropy (libp2p); this affects liveness but not safety or determinism.

3.2 Authorization Epochs via Verifiable Credentials and Status Lists

EQUACK manages permissions using verifiable, time-bounded credentials. It adopts the W3C Verifiable Credentials (VC) format for signed authorization tokens asserting a principal's rights over a specific interval (validFrom–validUntil). Credentials are tamper-evident and self-verifying; replicas independently confirm authenticity and validity [43].

Revocations use standard VC status lists—cryptographically signed bitstrings indicating whether each credential ID is revoked. A valid credential defines an authorization epoch; expiry or revocation ends it.

Trust evidence is carried in-band as TRUST events (issuer keys and status-list updates) and replayed into a local TrustView, anchored by a pinned, cross-signed genesis checkpoint. The log forms a distributed PKI: an offline node can verify at any timestamp that a user held a valid, non-revoked credential without contacting a central authority.

During replay, the system checks for a valid credential covering each event's timestamp; if found, the operation applies, otherwise it is skipped.

Trust operations (IssuerKey, IssuerKeyRevoke, StatusListChunk) build the TrustView incrementally during replay. When a VC must be checked, replicas parse the JWT, look up the issuer key in TrustView, verify the Ed25519 signature, check status-list revocation, and extract claims—all from in-band log state, with no online PKI lookups.

3.2.0.1 Status propagation without external endpoints.: Unlike the W3C Bitstring Status List specification, which assumes HTTP-accessible status endpoints, EQUACK propagates status-list updates as in-band TRUST events in the causal DAG. This sidesteps the need for always-online StatusList URLs: status freshness is bounded by event propagation latency rather than HTTP polling intervals. The trade-off is that status information is only as fresh as the latest TRUST event a replica has received. EQUACK's design is compatible with W3C Bitstring Status List at the data-format level; deployments with partial connectivity can supplement in-band propagation with periodic HTTP fetches when available.

3.3 Policy Semantics (Cedar) and Deny-Overrides Alignment

EQUACK uses a policy layer inspired by AWS's Cedar language (open-source ABAC). Policies may permit or forbid actions with a default deny stance; deny-overrides semantics ensure any matching forbid overrides all permits. The framework unifies RBAC and ABAC: roles are principal attributes, and Cedar's entity-attribute model supports hierarchical roles and contextual attributes. Policies are evaluated deterministically at each event's timestamp; identical inputs guarantee identical outcomes across replicas.

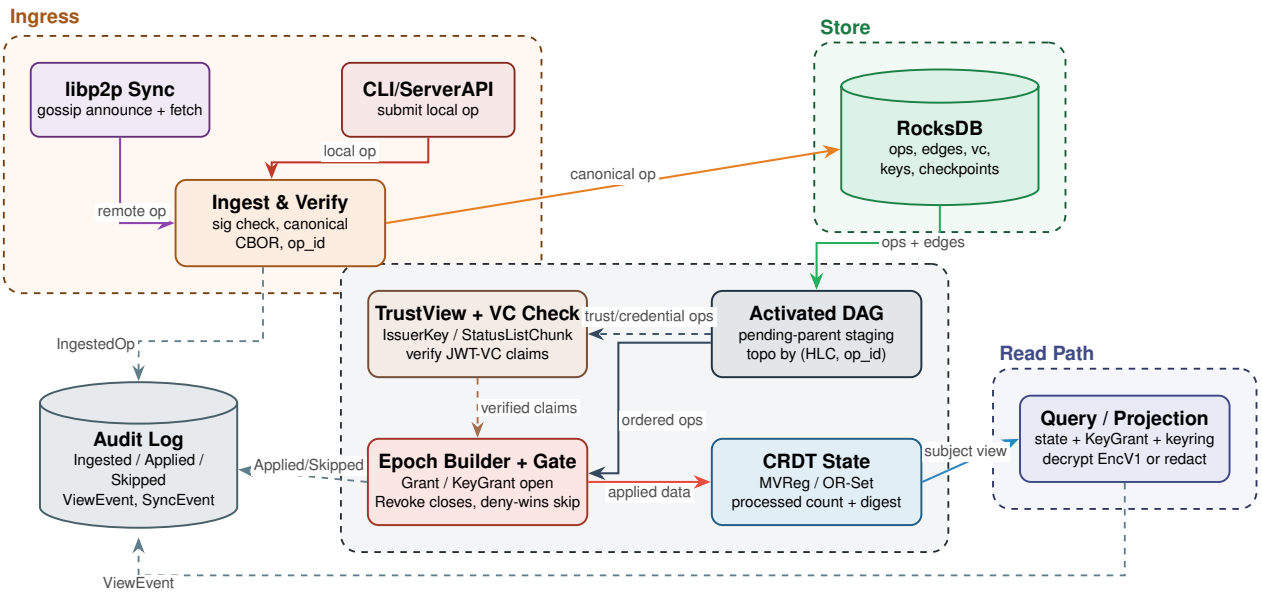

Fig. 1. EQUACK prototype overview showing the EQUACK four-phase lifecycle (Ingest, Verify, Activated DAG, Epoch/Gate, CRDT state, Query) and supporting infrastructure.

A REVOKE event deterministically terminates any authorization epoch opened by a prior or concurrent GRANT, so earlier permits cannot "win" due to message reordering. Importantly, revocation is not irreversible: a GRANT *after* a REVOKE in the replay order opens a fresh epoch (re-grant). For concurrent GRANT/REVOKE pairs, the deterministic linearization chooses a single order.

Replay Example. Our compliance fixture contains an in-band `IssuerKey`, credential-and-grant pairs for two writers, 50 concurrent pre-revoke writes from each writer, a merge point, a `REVOKE` for one writer, and 10 post-revoke writes from each writer. Replay first builds the in-band TrustView, verifies both credentials, and opens epochs for both writers. The later `REVOKE` closes only writer B's epoch in the deterministic replay order. As a result, all replicas apply both writers' pre-revoke writes, skip writer B's post-revoke writes, and continue applying writer A's writes.

3.3.0.1 Cedar property transfer under eventual consistency.: Cedar's formal verification [41] establishes decidability, totality, and soundness of authorization evaluation under a single consistent context—all entity data and policies are available and coherent at decision time. In EQUACK's eventually consistent setting, this assumption holds locally (each replica evaluates deterministically given its local state) but not globally during partitions: two replicas may reach different permit/deny outcomes for the same request if one has received a forbid policy or revocation that the other has not. The deny-wins replay mechanism restores global consistency after convergence: once all replicas hold the same event set, replay re-evaluates every operation under the converged policy set, and the final materialized state $s^\star$ satisfies Cedar's deny-overrides semantics. Cedar's verified properties thus hold for $s^\star$ but not for the transient local decisions made during partitions. Formal verification of the replay mechanism itself—specifically, that deterministic linearization plus deny-wins gating preserves the intended authorization invariants—is future work and would require extending Cedar's Lean 4 proofs or independent mechanized checking.

3.3.0.2 Deny-overrides as the combining algorithm.: The current model uses deny-overrides as the sole combining algorithm: any matching `forbid` policy overrides all `permit` policies. This is a deliberate safe default for multi-authority settings where no single authority should be able to override another authority's denial. Priority-based or temporal-precedence combining (e.g., last-writer-wins for policy updates) would require extending the epoch model to track policy precedence across authorities and is left as future work.

3.3.0.3 Entity hierarchy staleness.: Entity attributes used in Cedar policy evaluation (e.g., role assignments, group memberships) may be stale during partitions: a replica that has not yet received an attribute-update event evaluates policies against outdated entity state. Because attribute changes are events in the causal DAG, they are replayed deterministically on convergence, and the final state $s^\star$ reflects the correct attribute values. Transient local decisions based on stale attributes are corrected during replay.

Figure 3 shows this replay slice and the epochs and gate outcomes it induces.

3.4 Tamper-Evident Audit Trail

During replay, each replica emits a signed *audit record* per DATA operation indicating applied/skipped status and reason. Records are linked in an append-only hash chain; any inconsistency is detectable relative to a pinned chain head digest (e.g., cross-signed checkpoint). Because replay is deterministic, independent replicas with the same event set compute identical audit traces and can validate each other after reconciliation (R4).

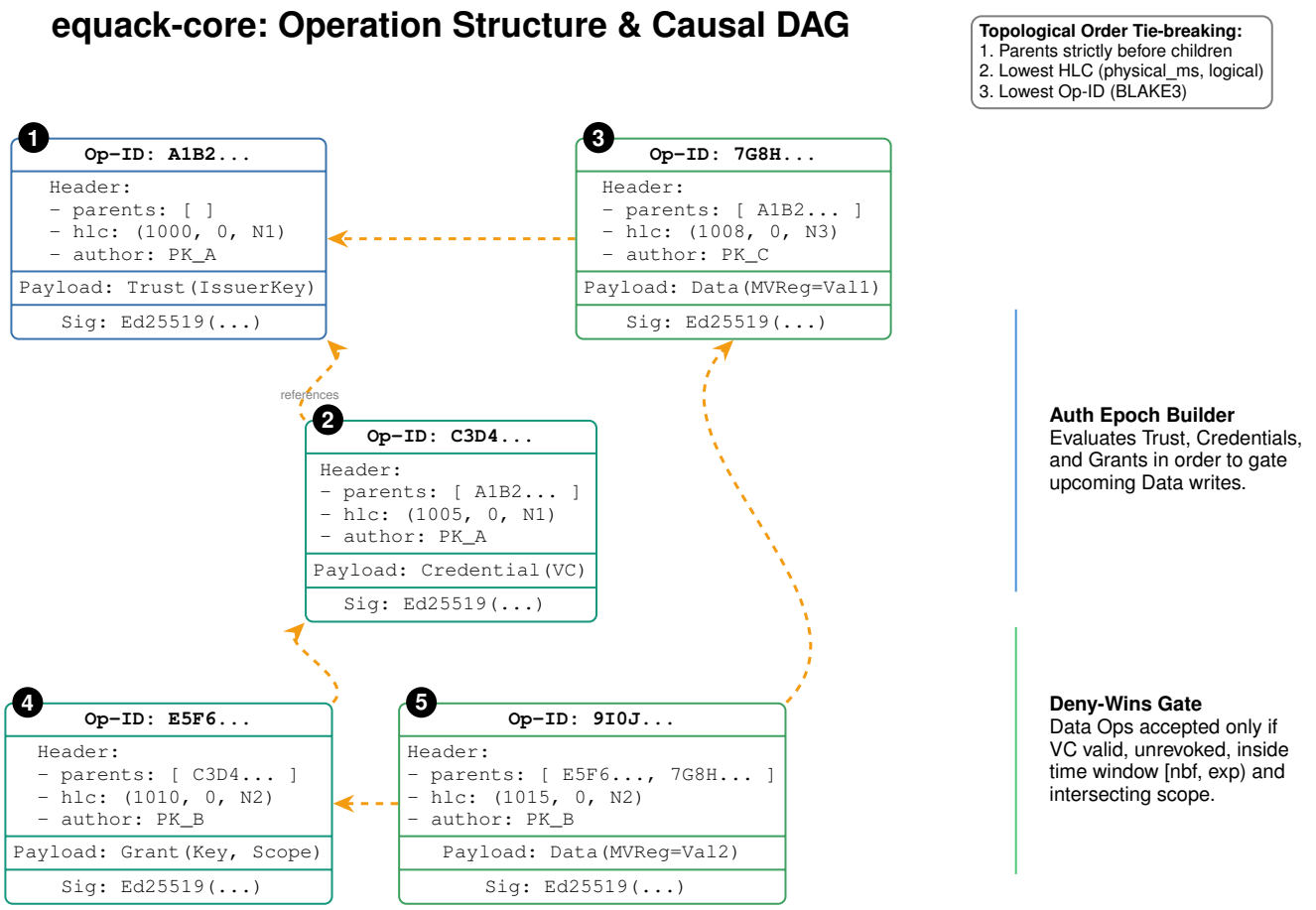

Fig. 2. Operation structure showing the OpHeader fields and payload variants, and the causal DAG deterministic topological sort breaking ties according to HLC and BLAKE3 Op-ID.

3.5 Confidentiality and Key Rotation

EQUACK supports an optional per-tag payload encryption layer. Each DATA operation carries ciphertext $c := \mathsf{Enc}_{K_\tau^{(v)}}(m)$ with key version v (XChaCha20-Poly1305). Ciphertext is replicated for convergence; access control is enforced at *query time* via the deny-wins gate, with unauthorized subjects receiving a redacted view. On revocation, a KeyRotate event advances to $K_\tau^{(v+1)}$, distributed in-band via KeyGrant records encrypted to authorized subjects' public keys. This provides *leaver security*: revoked subjects lacking $K_\tau^{(v+1)}$ cannot decrypt post-rotation ciphertexts. This component is experimental; we treat confidentiality results as indicative.

3.6 Formal Model and Guarantees

Let Subj be subjects (principals), Res resources, Act actions, and Attr attribute valuations. Physical time is measured in milliseconds as $(T_{\mathrm{ms}} = \mathbb{N}, \leq)$. Let NodeId be replica identifiers, and let hybrid logical clock timestamps range over $T_{\mathrm{hlc}} := T_{\mathrm{ms}} \times \mathbb{N} \times \mathsf{NodeId}$ ordered lexicographically. Let $\mathrm{phys} : T_{\mathrm{hlc}} \to T_{\mathrm{ms}}$ project the physical millisecond component. We use T_{ms} for credential validity intervals and status freshness, and T_{hlc} (via lexicographic order) only for deterministic linearization. Let H be a collision-resistant hash and $\mathsf{Sig}_k(\cdot)$ a secure signature scheme.

Verifiable credentials and trust. A verifiable credential (VC) is a tuple

$$c = \langle s, \alpha, V, \mathrm{nb}, \mathrm{na}, \sigma_I \rangle$$

where $s \in \mathsf{Subj}$, $\alpha \in \mathsf{Attr}$ (roles/scopes), V issuer identifier, and $\mathrm{nb}, \mathrm{na} \in \mathbb{T}$ are not-before/after bounds. σ_I is a signature under issuer key K_V. A status list is a signed, time-versioned snapshot published by issuer V that maps credential identifiers to $\{\mathsf{good}, \mathsf{revoked}\}$. We model status evidence as a sequence of TRUST events that carry status-list updates (or chunks). Let SL_V^i denote the status map materialized by replaying all TRUST events with $\mathrm{pos}(e) \leq i$ and taking the latest published status for each credential id. When validating a credential for an event e, we query $\mathsf{SL}_{\mathrm{iss}(c)}^{\mathrm{pos}(e)}(\mathrm{id}(c))$. A *TrustView* event carries (i) issuer keys $\{K_V\}$ and (ii) status-list updates, recorded in-band so replicas can validate VCs offline using the latest status snapshot up to t in the replay order.

Events and log. Events are partitioned as

$$\mathsf{Ev} = \mathsf{DATA} \uplus \mathsf{GRANT} \uplus \mathsf{REVOKE} \uplus \mathsf{TRUST} \uplus \mathsf{KEY}.$$

The subset KEY contains key-management events (e.g., KEYROTATE, KEYGRANT) that update per-tag key state used for query-time decryption. Each event $e \in \mathsf{Ev}$ has an *unsigned* header $h_0(e)$ containing domain fields, causal

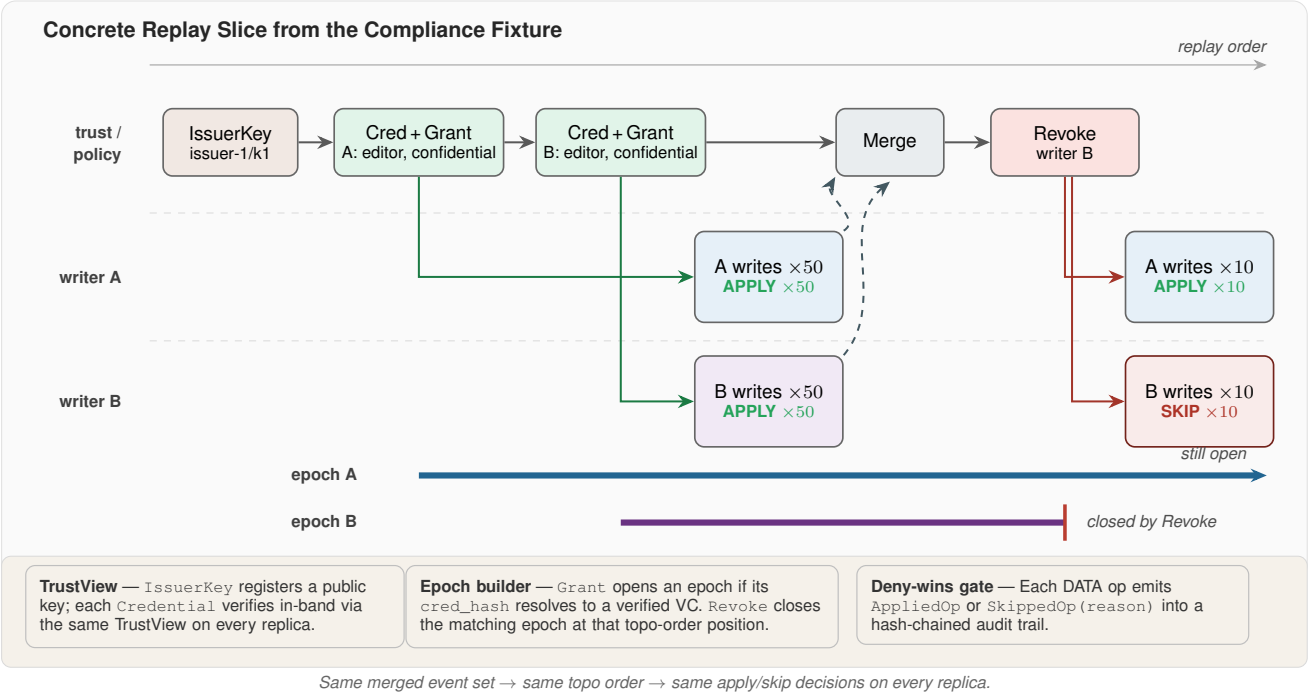

Fig. 3. Deterministic replay slice from the compliance fixture. An in-band `IssuerKey` seeds TrustView, verified `Credential`+`Grant` pairs open epochs for writers A and B, and `Revoke(B)` closes only B's epoch. The deny-wins gate therefore applies both writers' pre-revoke data, skips B's post-revoke data, continues applying A's post-revoke data, and emits matching audit events.

parents $\mathsf{par}(e) \subseteq \mathsf{Ev}$, a hybrid logical clock $\mathrm{hlc}(e) \in \mathbb{T}$ assigned by the producing replica, the producer public key pk_r, and the payload (or a payload hash). We define the event identifier as

$$\mathsf{op_id}(e) := \mathsf{H}(\mathrm{domain} \,\|\, \mathrm{CBOR}(h_0(e))).$$

The producing replica signs this identifier:

$$\sigma_e := \mathrm{Sig}_{\mathrm{sk}_r}(\mathsf{op_id}(e)).$$

Signing $\mathsf{op_id}(e)$ is sufficient because $\mathsf{op_id}(e)$ commits to the canonical bytes of $h_0(e)$ under collision resistance of H. The full header is $h(e) := (h_0(e), \sigma_e)$. This avoids any circularity: $\mathsf{op_id}(e)$ is derived from the unsigned header, and σ_e covers $\mathsf{op_id}(e)$. The *signed, hash-linked op log* is an append-only bag of events with edges $e' \to e$ for $e' \in \mathsf{par}(e)$. The induced graph $G = (\mathsf{Ev}, E)$ must be a DAG; this causal DAG defines the partial order $\prec$ as the transitive closure of E.

Deterministic linearization. Let the deterministic key be

$$k(e) := \big(\mathrm{hlc}(e), \mathsf{op_id}(e)\big),$$

ordered lexicographically. Define the replay sequence $\langle e \rangle_{\triangleleft}$ as the output of Algorithm 1 applied to the causal DAG G. We then define $e_i \triangleleft e_j$ iff e_i appears before e_j in $\langle e \rangle_{\triangleleft}$.
Let $\langle e \rangle_{\triangleleft} = [e_0, \ldots, e_{n-1}]$ be the output of Algorithm 1. Define the deterministic replay position $\mathrm{pos}(e_i) := i$.

If two events e_A and e_B are concurrent and both are ready (zero indegree) at some step, Algorithm 1 selects the smaller key: if $\mathrm{hlc}(e_A) < \mathrm{hlc}(e_B)$ then e_A is chosen first; if equal, the smaller $\mathsf{op_id}$ is chosen.

Algorithm 1 Deterministic linearization of a causal DAG

Require: DAG $G = (V, E)$, key $k(e) = (\mathrm{hlc}(e), \mathsf{op_id}(e))$
Ensure: Sequence L that is a deterministic linear extension of $\prec$

```
indeg[v] ← |{(u, v) ∈ E}| for all v ∈ V
Q ← min-priority queue ordered by k(·)
for all v ∈ V with indeg[v] = 0 do
    Q.push(v)
end for
L ← [ ]
while Q not empty do
    v ← Q.pop_min()
    append v to L
    for all (v, w) ∈ E do
        indeg[w] ← indeg[w] − 1
        if indeg[w] = 0 then
            Q.push(w)
        end if
    end for
end while
return L
```

Authorization epochs. Epochs are computed deterministically by replaying non-DATA events in $\langle e \rangle_{\triangleleft}$. A GRANT event g at position $p_g := \mathrm{pos}(g)$ that references a valid credential opens an epoch record $\mathsf{ep} = \langle s, a, \mathrm{scope}, p_{\mathrm{start}}{=}p_g, p_{\mathrm{end}}{=}\infty, \mathrm{nb}, \mathrm{na} \rangle$. A REVOKE event r at position $p_r := \mathrm{pos}(r)$ closes matching open epochs by setting $p_{\mathrm{end}} := \min(p_{\mathrm{end}}, p_r)$. We write $\mathsf{ep} \in \mathsf{Epochs}(s, a)$ for epochs associated with subject s and action a.

For subject s, scope $a \in \mathsf{Act}$, and event e, define the authorization predicate

$$\mathsf{Auth}(s,a,e) := \exists c \text{ s.t. } \mathrm{phys}(\mathrm{hlc}(e)) \in [\mathrm{nb}, \mathrm{na}) \ \wedge \ c \text{ grants } (s,a) \\ \wedge \ \mathsf{SL}^{\mathrm{pos}(e)}_{\mathrm{iss}(c)}(\mathrm{id}(c)) = \mathsf{good}.$$

Status-list TTL is a freshness hint for operational policy; our determinism guarantees assume replicas replay the same TrustView history, while deployments may additionally deny decisions when status evidence is older than a configured threshold.

Policy engine and deny-wins gate. Let $\mathcal{P}$ denote the Cedar policy, modeled as a total function

$$\mathcal{P} : \mathsf{Subj} \times \mathsf{Act} \times \mathsf{Res} \times \mathsf{Attr} \to \{\mathsf{allow}, \mathsf{deny}\},$$

with *deny-overrides* semantics. For a DATA event e with tuple (s, a, r, α_e), define:

$$\mathsf{Gate}(e) := \Big(\exists\, \mathsf{ep} \in \mathsf{Epochs}(s,a) \text{ s.t.} \\ \mathrm{pos}(e) \in [p_{\mathrm{start}}, p_{\mathrm{end}}) \wedge \\ \mathrm{phys}(\mathrm{hlc}(e)) \in [\mathrm{nb}, \mathrm{na})\Big) \\ \wedge \ \mathcal{P}(s,a,r,\alpha_e) = \mathsf{allow}.$$

If *any* deny applies, $\mathcal{P}$ returns deny and the DATA op is skipped (deny-wins).

CRDT layer and state. Let $(\mathcal{S}, \bullet)$ be the state space with a deterministic update function $U : \mathcal{S} \times \mathsf{DATA} \to \mathcal{S}$. We assume the DATA payloads target CRDTs such as MVReg and $\mathrm{OR-Set}$ whose effect under the same total order is deterministic. Given initial state s_0, define *deterministic replay* over the sequence $\langle e \rangle_{\lhd}$:

$$\mathsf{Replay}(s_0, \langle e \rangle_{\lhd}) := \mathrm{fold}\Big(s_0,\ \lambda(s,e). \\ \begin{cases} U(s,e) & \text{if } e \in \mathsf{DATA} \ \wedge \ \mathsf{Gate}(e) \\ s & \text{otherwise} \end{cases}\Big).$$

The *materialized state* is $s^\star := \mathsf{Replay}(s_0, \langle e \rangle_{\lhd})$. Checkpoints store $(\mathsf{H}(s^\star), \mathrm{meta})$ periodically together with the append-only event store.

Audit trail. Define the audit stream as a sequence of records

$$\mathsf{Aud}(i) = \langle e_i,\ \mathsf{Gate}(e_i),\ \mathsf{H}(s_i)\rangle, \\ \mathrm{ch}_0 := \bot, \quad \mathrm{ch}_{i+1} := \mathsf{H}(\mathrm{ch}_i \,\|\, \mathsf{Aud}(i)),$$

signed by the auditing replica (producer) to yield a tamper-evident chain (ch_i) of applied/skipped decisions.

Confidentiality and key rotation. Let Tag denote data tags; each tag τ has a versioned symmetric key $K_\tau^{(v)}$. A DATA payload m is stored as (c, v) where $c := \mathsf{Enc}_{K_\tau^{(v)}}(m)$. Ciphertexts are replayed and replicated for convergence; authorization controls *decryption* at query time. On a KeyRotate event for τ at time t_r, the current key version advances to $v{+}1$ and subsequent writes use $v{+}1$; KeyGrant records distribute $K_\tau^{(v+1)}$ only to subjects authorized to read τ after t_r.

Events are disseminated via secure anti-entropy gossip; missing parents are fetched until G is complete. This affects *when* a replica can replay but not *what* $s^\star$ is, once the same set of events is present.

3.6.0.1 Pre-convergence safety window.: Let r be a revocation event created at physical time t_r on some replica. Define the *safety window* $W(r)$ as the interval $[t_r, t_{\mathrm{heal}}]$ where t_{heal} is the earliest time at which all correct replicas have received r via anti-entropy. During $W(r)$, replicas that have not yet received r may accept operations from the revoked principal under stale authorization evidence. Under the bounded partition assumption (Section 2.3, item 4), $W(r) \le \Delta_{\mathrm{partition}} + \Delta_{\mathrm{propagation}}$ is finite.

EQUACK does not prevent tentative authorization during $W(r)$; it guarantees that the materialized state $s^\star$ computed after reconciliation reflects r. Specifically, any DATA event e from the revoked subject with $\mathrm{pos}(e) \ge \mathrm{pos}(r)$ in the deterministic order $\lhd$ is skipped by the deny-wins gate, regardless of which replica originally accepted e. This provides *retroactive revocation safety*: the final digital state is consistent with the revocation, even though individual replicas may have temporarily acted on stale evidence. In contrast, Zanzibar's Zookie mechanism [39] provides real-time revocation at the cost of availability during partition (consistent with CAP theorem constraints). EQUACK trades real-time revocation for availability, repairing safety on merge.

3.6.1 Main Theorems (Formal Guarantees)

Requirements vs. formal guarantees. R1–R6 in Section 2.4 are *system-level requirements* (deployability and security objectives) for offline-first, multi-stakeholder settings. The theorems below are *formal guarantees* about EQUACK's deterministic replay semantics and audit/confidentiality mechanisms under the model assumptions. The gap analysis (Table 1) maps R1–R6 to paradigm coverage; the theorems below formalize the guarantees, and the evaluation (Section 5) provides empirical evidence.

Theorem 1 (Determinism). *For any DAG G and initial state s_0, the order $\lhd$ is unique and $\mathsf{Replay}(s_0, \langle e \rangle_{\lhd})$ yields a unique $s^\star$.*

Algorithm 1 is Kahn's topological sort with a deterministic min-heap over the ready set. For a fixed DAG G and a total key order k, it produces a unique linear extension $\langle e \rangle_{\lhd}$. Replay is a pure fold over this sequence with a deterministic update function, hence yields a unique $s^\star$.

Theorem 2 (Convergence). *Fix bootstrap anchors B consisting of a pinned, cross-signed genesis checkpoint digest (and the stakeholder public keys used to validate it). Let two replicas start from the same anchors B and ingest the same multiset of well-formed events (including* TRUST *and* KEY *events). Then their materialized states are byte-identical: $s_1^\star = s_2^\star$.*

Both replicas begin from the same anchored initial trust/policy state implied by B, ingest the same event set, and thus compute the same causal DAG G. Algorithm 1 yields the same $\lhd$ order, hence the same $\mathrm{pos}(\cdot)$ assignments, the same epoch construction, and the same deny-wins gate decisions. Applying the same ordered sequence of authorized DATA updates under deterministic U yields identical $s^\star$.

Theorem 3 (Policy-Safety, deny-wins). *Every effect in $s^\star$ originates from a* DATA *event e such that* $\mathsf{Gate}(e)$ *held at position $\mathrm{pos}(e)$. If any deny applies (policy or missing epoch), e has no effect.*

By definition of Replay: unauthorized DATA are skipped; deny overrides return deny, hence no state transition occurs.

Theorem 4 (Audit tamper-evidence). *Let ch_N be a pinned audit-chain head digest for the first N records (obtained out-of-band or via cross-signing). If the audit chain verifies and the computed head equals ch_N, then any modification to the first N audit records (deletion, insertion, reordering, or bit-flip) is detectable with overwhelming probability.*

The head digest is a collision-resistant hash of the prefix. Any change to the prefix changes the computed head unless the adversary finds a collision/second preimage or forges a signature.

Theorem 5 (Leaver security). *Fix a tag τ and a rotation event at time t_r that advances to key version $v{+}1$. Assume IND-CPA encryption and that $K_\tau^{(v+1)}$ is delivered only to subjects authorized for read_τ at times $\geq t_r$. Then any subject not authorized for read_τ at times $\geq t_r$ cannot decrypt ciphertexts produced under version $v{+}1$, except with negligible probability.*

By assumption, unauthorized subjects do not obtain $K_\tau^{(v+1)}$. Under IND-CPA security, ciphertexts under an unknown key reveal no information about plaintext beyond negligible probability.

4 Implementation

EQUACK-core is a Rust library organized into four crates that map directly to the conceptual model:

- `core`: Operations, DAG construction, deterministic replay, policy evaluation (Cedar), VC parsing and verification, and CRDT state (MVReg, OR-Set). This crate implements the formal model from Section 3.6.
- `store`: RocksDB persistence, checkpoint creation and restoration, and audit chain management. Implements the checkpoint and audit mechanisms.
- `net`: libp2p-based gossip dissemination and anti-entropy synchronization. Implements the liveness layer.
- `cli`: Command-line interface for workload generation, log manipulation, and experiment execution.

The implementation uses Ed25519 for signatures, BLAKE3 for hashing, XChaCha20-Poly1305 for encryption, and SHA-256 for status list bitstrings. Operations are serialized in CBOR. The prototype comprises approximately 8,500 lines of Rust (excluding tests).

Table 3 maps conceptual elements to implementation modules.

TABLE 3
Mapping from conceptual model to implementation modules.

Conceptual Element	Implementation Module
Causal DAG $(G, \prec)$	`core::dag`
Deterministic order $(\lhd)$	`core::replay::topo_sort`
Authorization epochs	`core::epoch`
Deny-wins gate (Gate)	`core::policy::gate`
TrustView assembly	`core::trust`
VC verification	`core::vc`
CRDT state $(\mathcal{S}, U)$	`core::crdt`
Audit chain	`store::audit`
Checkpoints	`store::checkpoint`
Per-tag encryption	`core::crypto`
Gossip/sync	`net::gossip`

5 Evaluation

We evaluate EQUACK through deterministic experiments that provide empirical evidence for the formal guarantees (R2–R5) and quantify replay cost. All experiments use fixed seeds and a deterministic environment (`TZ=UTC`, `SOURCE_DATE_EPOCH=1`), and assume offline operation without continuous global consensus (R1).

5.1 Methodology

Experiments ran on a single Linux x86_64 workstation using Rust 1.85 in release mode. Multi-node convergence (E12) uses in-process deterministic simulation; partition behavior is validated via offline log merges. State digests are computed as $\mathrm{blake3}(\mathrm{DOMAIN} \,\|\, \mathrm{canon}(\mathit{state}))$, where $\mathrm{canon}(\cdot)$ is a deterministic byte serialization with stable key ordering.

We use three synthetic workloads (Table 4) and two policy-focused CBOR logs. The *hb-chain* workload is a single writer appending sequentially to one key, giving a baseline with no concurrency or policy overhead. The *concurrent* workload has eight writers modifying the same key, stressing the MVReg CRDT's conflict resolution. The *offline-revocation* workload mixes writes with mid-workload revocations, forcing epoch recalculation during replay. Two additional logs exercise deny-wins behavior: `e3-policy.cbor` (grant, 500 writes, revoke, 500 writes) and `e4-policy.cbor` (multi-issuer grants, revoke, post-revoke writes). For the partition scenario, logs `partition-A.cbor` and `partition-B.cbor` were generated and merged to model revocation propagation on reconnection.

TABLE 4
Synthetic Workload Characteristics

Workload	Writers	Policy Ops	Key Dist.	Purpose
hb-chain	1	0%	Single	Baseline
concurrent	8	0%	Single	CRDT stress
offline-revocation	8	Mixed	Single	Policy overhead

5.2 Correctness

Convergence (E1, R2): A 10,000-op concurrent-workload log was permuted into 100 valid orderings (respecting causal dependencies) and each was replayed independently. All 100 produced the same digest (`cbab89ee...b7f493b2`), with mean replay time 615 ms ($\sigma = 18$ ms).

Deny-wins revocation (E3, R3): User U was granted write access, performed 1,000 writes, and had the credential revoked after write 500. The 1,004-op log (1,000 Data + IssuerKey, Credential, Grant, Revoke) was replayed: the first 500 writes applied, the remaining 500 skipped. Digest: `8da8b49e...46633fbf`.

Revocation under partition (E11, R3): In partition A a user is revoked; partition B, which has not yet received the revocation, contains further writes from the same user. After merging both logs and replaying, the 50 pre-revocation writes (both partitions) were applied and all 10 post-revocation writes from partition B were skipped (merged digest: `69fae61e...1a2986c1`). This is the scenario EQUACK

was designed for. Figure 4 visualizes this repair-on-heal semantics: a partitioned replica may locally accept stale writes, but those writes are not part of the final digital state unless they remain authorized after the union log is replayed.

Multi-authority conflict (E4, R5): Three issuers (I_1, I_2, I_3) with overlapping authority grant user U the "editor" role at t_1, t_2; I_3 revokes at $t_3 > t_2$. The resulting 128-op log was injected into 10 replicas in random orders. All converged to the same digest (`f2fbb024...1103c2b2`): writes during $[t_1, t_3)$ accepted, writes after t_3 skipped.

Audit integrity (E5, R4): An audited log was generated and the chain head digest ch_N was recorded. Three tamper modes were applied: (1) a byte flip in one record's payload, (2) deletion of one record from the pinned prefix, and (3) a swap of two adjacent records. Chain verification against ch_N succeeded on the untampered log and failed on all three mutations, detecting each at the exact tampered record.

Cross-platform determinism (E2): A 127-op CBOR fixture exercises all event categories (Data, Grant, Revoke, Trust) and includes true concurrency. A CI test replays it and asserts the final digest equals a hardcoded constant (`4ea9684a...bdc2b9`, blake3). Any replay-engine change that alters the digest breaks the build, following the approach of cross-environment consistency verification [44]. Full cross-platform validation (aarch64, macOS, Windows) awaits CI infrastructure.

Multi-node sync (E12): In an in-process simulation, three nodes each generated 1,000 ops independently, then synced; all converged. A second scenario partitioned node C while A issued a Revoke targeting C. After healing and full sync, all three nodes held the same digest, with C's post-revoke ops gated out (Table 5).

TABLE 5
Multi-node sync convergence (E12)

Scenario	Ops	Sync (ms)	Bytes	Msgs	Replay (ms)
No partition	3,000	13	2.5 MB	6,000	411
Partition + heal	3,000	7	1.3 MB	3,201	368

5.3 Performance

Replay scaling (E6): Hb-chain logs at six sizes (10K–500K ops) were each replayed in 5 trials, measuring both full replay and incremental replay from a 90% checkpoint (Table 6).

TABLE 6
Replay time vs. log size (mean $\pm$ std, 5 trials)

Ops	Full (ms)	Incr. (ms)	Speedup
10,000	949 ± 395	98 ± 37	9.7×
20,000	$1{,}294 \pm 79$	147 ± 21	8.8×
50,000	$3{,}328 \pm 233$	404 ± 54	8.2×
100,000	$6{,}071 \pm 92$	708 ± 86	8.6×
250,000	$13{,}689 \pm 1{,}814$	$1{,}644 \pm 232$	8.3×
500,000	$26{,}616 \pm 2{,}091$	$3{,}137 \pm 310$	8.5×

Full replay scales linearly (≈0.052 ms/op, $R^2 > 0.99$), giving ≈19K ops/s sustained throughput on the hb-chain workload. Incremental replay from a 90% checkpoint yields 8–11× speedup across all sizes; the factor is stable because checkpoint restore is $O(1)$ and the suffix cost is proportional to 10% of the log. Checkpoint creation overhead is under 10 ms.

Throughput by workload (E7): Throughput varies with workload complexity (Table 7). Linear chains achieve the highest rate; concurrent writes reduce throughput because of MVReg conflict resolution; policy-heavy workloads show modest epoch-construction overhead.

TABLE 7
Replay throughput by scenario

Scenario	Throughput (ops/s)
hb-chain (linear)	45,000
concurrent (8 writers)	7,000
offline-revocation	28,000

Storage and memory (E8, E9): CBOR log size grows linearly at ≈430 bytes/op; RocksDB write amplification brings the on-disk cost to ≈800 B/op (Table 8). Checkpoint state remains under 200 bytes for the single-key workload. Peak RSS during replay grows at ≈1.7 KB/op (Table 9), dominated by the in-memory DAG and CRDT state.

TABLE 8
Storage growth vs. op count

Ops	CBOR Log	Ckpt State	Store Total	B/op
10,000	4.1 MB	122 B	7.7 MB	804
50,000	20.6 MB	118 B	37.8 MB	792
100,000	41.4 MB	127 B	75.7 MB	793
500,000	208.9 MB	130 B	555.8 MB	1,165

TABLE 9
Peak RSS during replay

Ops	Peak RSS (MB)	Replay (ms)
20,000	31	1,331
100,000	131	6,571
250,000	421	34,622

Confidentiality overhead (E13, E14): Per-tag encryption adds ∼6.5% replay overhead (76.8 vs. 81.8 ms on 100 writes) with 40 bytes/payload ciphertext expansion. Key rotation costs ∼78 ms per KeyRotate event. These are small-scale measurements; larger logs are needed to separate per-op cost from startup overhead.

5.4 Limitations and Threats to Validity

Disconnected nodes cannot learn of revocations until they sync [45]. The vulnerability window equals partition duration: operations written by a revoked user are invalidated on reconnection, but not before. The append-only log grows monotonically [46], and the full log must be retained for audit. Log compaction with verifiable snapshots [47] could bound storage but remains future work. Long-offline nodes face a proportionally large replay on reconnection;

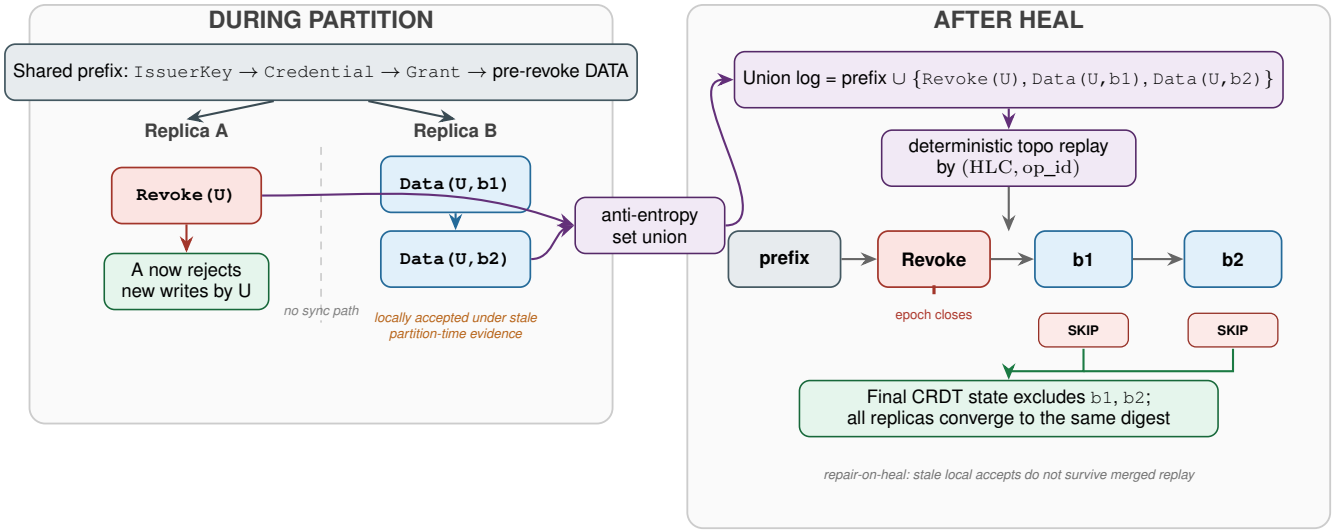

Fig. 4. Partition-heal semantics of deterministic deny-wins replay. Replicas share a common prefix before partition. During the partition, replica A receives `Revoke(U)` while replica B, still missing that event, locally accepts subsequent writes by U under stale evidence. After anti-entropy, all replicas replay the union log in the same deterministic order; the revoke closes U's epoch, the post-revoke writes are skipped, and every replica converges to the same final state digest.

checkpoints help but do not eliminate this. Per-tag encryption protects payloads, yet operation structure, timestamps, and producer keys remain visible [48]; full metadata privacy would require anonymous credentials, ABE, or zero-knowledge proofs [49]–[51]. Revocation also cannot undo physical-world actions already taken (e.g., an actuator that has fired). Applications needing staged commit must handle this themselves.

Regarding validity: E6 covers six log sizes with five trials each, but E8 and E9 use fewer sizes and a single workload. All numbers come from one Linux x86_64 machine, and E12 uses in-process simulation rather than real network transport. The workloads are synthetic; validation on production traces is future work. E2 determinism is verified on one platform; cross-platform validation (aarch64, macOS, Windows) awaits CI infrastructure.

6 Discussion

The partition-duration trade-off: EQUACK's deny-wins gate repairs authorization state after partition heal, but it cannot prevent unauthorized effects during the partition itself. A revoked user's writes are retroactively invalidated when logs merge, yet any physical-world consequences of those writes (actuator commands, parameter changes applied to running equipment) persist. The vulnerability window is exactly the partition duration. For short disconnections—minutes to a few hours, typical of scheduled maintenance windows—operators can defer safety-critical actuations until sync confirms authorization. For day-long partitions in remote deployments, the exposure is more serious and may require application-layer compensating controls (staged commit, human-in-the-loop confirmation) that EQUACK does not prescribe. The guarantees are about the *digital* state; the physical world cannot be rewound.

Scalability scope: The evaluation numbers bound where EQUACK is and is not a good fit. Memory grows at ≈1.7 KB/op (Table 9), so a 256 MB edge device can hold roughly 150K operations before paging—enough for weeks of maintenance logs on a single controller, but not for high-frequency telemetry streams. Replay throughput drops from 45K ops/s on linear chains to 7K ops/s with eight concurrent writers (Table 7), a 6.4× reduction driven by MVReg conflict resolution. Real deployments will have more keys and more writers; the scaling behavior under those conditions is untested. The append-only log compounds this: at ≈430 bytes/op in CBOR, a million-operation log occupies ∼430 MB before RocksDB write amplification. Without log compaction, long-lived deployments will eventually need to archive and prune, a mechanism the current design defers to future work (verifiable snapshots [47]). These constraints mean EQUACK fits authorization-rate workloads (access decisions per minute, not per millisecond) on devices with at least modest storage—industrial controllers, maintenance laptops, gateway nodes—rather than high-throughput data-plane enforcement.

Deny-wins conservatism: The deny-wins semantics are deliberately conservative: when a GRANT and REVOKE are concurrent, the revoke wins. This is the safe default for settings where over-granting is riskier than over-denying, such as safety-parameter updates on shared equipment. But it has practical consequences. An erroneous revocation during a partition retroactively voids all subsequent work by that subject, even if the revocation was itself a mistake. The re-grant mechanism (Section 3.6) allows recovery—a new GRANT after the REVOKE opens a fresh epoch—but this requires a second authority action that may not be timely during disconnection. Systems where availability is more important than safety (e.g., collaborative editing in low-risk domains) might prefer a *permit-wins* or *last-writer-wins* policy for certain operation types. EQUACK's

Cedar policy layer could, in principle, express such rules, but the epoch/gate mechanism currently hard-codes deny-wins at the replay level. Making this configurable per scope or resource type is a straightforward extension, though it would complicate the convergence argument.

In-band trust: benefits and costs: Carrying issuer keys and status-list updates as replicated log events eliminates the need for online PKI lookups, which is what makes offline VC verification possible. The cost is that every replica stores all trust material, and the TrustView grows with each issuer key registration and status-list chunk. In the evaluated workloads this overhead is negligible (a handful of trust events per log), but a deployment with frequent key rotations across many issuers would see meaningful log growth from trust material alone. The pinned genesis checkpoint anchors the entire trust chain; its creation is a cross-signing ceremony that the current paper treats as a precondition rather than specifying its protocol. For multi-organization deployments, the governance of this ceremony—who signs, how keys are distributed, how the genesis is updated when organizations join or leave—is an open design problem that we have not addressed.

Comparison scope: The gap analysis (Table 1) and mechanism comparison (Table 2) evaluate EQUACK against paradigm categories rather than specific systems, because no existing system combines CRDT-based replication with in-band VCs and a deny-wins gate. A fair question is how EQUACK compares to a CRDT with application-level access-control logic bolted on. The difference is where enforcement runs: application-level checks execute at ingest time and cannot retroactively invalidate operations accepted under stale evidence during a partition; EQUACK's replay-time gate can, which is what provides R3. The absence of a directly comparable system makes head-to-head benchmarking infeasible and means the performance numbers lack an external reference point—whether 45K ops/s or 7K ops/s is adequate depends entirely on the deployment.

Pre-convergence safety window: As formalized in the safety window definition ($W(r) \leq \Delta_{\text{partition}} + \Delta_{\text{propagation}}$), EQUACK provides retroactive revocation safety, not real-time revocation. During the safety window, replicas that have not received a revocation may accept operations under stale evidence. This is an inherent trade-off of availability-first designs under CAP constraints: Zanzibar's Zookie provides real-time consistency at the cost of availability, while EQUACK provides availability at the cost of a bounded safety window. The guarantee is that the final digital state $s^\star$ is always consistent with the revocation, regardless of the duration of $W(r)$.

Cedar property transfer: Cedar's verified properties (decidability, totality, soundness) hold for the converged state $s^\star$ but not for transient local decisions during partitions. This is a fundamental limitation of any eventually consistent system that embeds a policy engine designed for single-context evaluation. The replay mechanism restores the invariant, but the transient violation window cannot be eliminated without sacrificing availability. Mechanized verification of the replay-gate composition—showing that deny-wins gating over a deterministic linearization preserves the intended authorization invariants—would strengthen this claim and is identified as future work.

Prototype scope and limitations: The EQUACK prototype implements the full conceptual model but with simplifications in several areas. VC validation supports the core W3C data model fields (issuer, subject, validity period, status) but not the full VC Data Model 2.0 specification (e.g., no JSON-LD processing, no selective disclosure). Key management uses X25519 for key agreement but does not implement a full key ceremony protocol; the genesis checkpoint is assumed to be pre-shared. Cedar policy evaluation covers the core permit/forbid/deny-overrides semantics but not Cedar's full type system or schema validation. The libp2p gossip layer uses basic flooding rather than a production-grade protocol with backpressure or prioritization.

From prototype to deployment: Several gaps remain between the EQUACK prototype and a production-ready system. The evaluation uses synthetic workloads on a single machine; real deployments involve heterogeneous devices, unreliable links, and bursty operation patterns that synthetic logs do not capture. E12 simulates multi-node sync in-process rather than over a real network, so sync timing under packet loss and bandwidth constraints is unknown. Cross-platform determinism (E2) is verified on Linux x86_64 only; a single non-determinism bug in serialization or floating-point handling on another architecture would break convergence. The confidentiality layer is explicitly experimental: overhead was measured on 100 writes, far too few to characterize per-operation cost at scale. Finally, the formal guarantees (Theorems 1–5) are stated and argued but not mechanically verified; a Coq or Isabelle formalization of the replay and gate logic would close the gap between the model and the implementation.

7 Conclusion

This paper addressed the offline enforcement problem: how can independent authorities grant and revoke permissions for shared resources that operate during disconnection, while converging to a consistent and auditable authorization state? EQUACK operationalizes this using a causal event DAG with deterministic replay: all events influencing authorization are recorded as signed, hash-linked operations; replicas merge by set union and replay under a deny-wins gate that skips unauthorized effects.

The evaluation provides empirical evidence consistent with the formal guarantees. Replicas converged across 100 randomized orderings (E1); post-revocation effects were suppressed, including across partition boundaries (E3, E11); audit tampering was detected (E5); and multi-issuer conflicts resolved deterministically (E4). Replay throughput reaches 45k ops/s for linear chains with 8–11× checkpoint speedup (E6, E10). Three-node sync convergence is confirmed under partition-heal conditions with deny-wins revocation (E12). Encryption overhead is modest (∼6.5%).

The evaluation is primarily single-machine with synthetic workloads; E12 uses in-process simulation. Cross-platform determinism (E2) is verified on Linux x86_64; multi-platform CI is deferred. The model assumes eventual delivery, correctly managed trust inputs, and uncompromised key material. Revocation cannot undo irreversible physical-world actions. As formalized in the pre-convergence safety window analysis, EQUACK provides

retroactive revocation safety—the converged state $s^\star$ always reflects revocations—but does not prevent transient authorization under stale evidence during partitions. Cedar's verified properties hold for $s^\star$ but not for transient local decisions.

Immediate priorities include multi-machine evaluation with network-realistic partition injection, realistic workloads (IoT fleet management, collaborative editing), and baseline comparisons against standalone policy engines (Cedar, OPA) and Zanzibar-family systems. Mechanized verification of the replay-gate composition (TLA$^+$ or Lean) would close the gap between the formal model and the implementation. Longer-term directions include extending the epoch model to support configurable combining algorithms beyond deny-overrides, governance frameworks for multi-authority coordination, log compaction with verifiable snapshots, Jepsen-style invariant testing, and privacy-preserving techniques (anonymous credentials, attribute-based encryption, zero-knowledge proofs). Real-world deployment in industrial control systems and federated research settings is essential for validating the model under realistic conditions.

References

[1] I. Mistry, S. Tanwar, S. Tyagi, and N. Kumar, "Blockchain for 5g-enabled iot for industrial automation: A systematic review, solutions, and challenges," *Mechanical Systems and Signal Processing*, vol. 135, p. 106382, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S088832701930603X

[2] H. Liu, D. Han, and D. Li, "Fabric-iot: A blockchain-based access control system in iot," *IEEE Access*, vol. 8, pp. 18 207–18 218, 2020.

[3] A. Hasselgren, K. Kralevska, D. Gligoroski, S. A. Pedersen, and A. Faxvaag, "Blockchain in healthcare and health sciences—a scoping review," *International Journal of Medical Informatics*, vol. 134, p. 104040, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S138650561930526X

[4] K. Gai, Y. She, L. Zhu, K.-K. R. Choo, and Z. Wan, "A blockchain-based access control scheme for zero trust cross-organizational data sharing," *ACM Trans. Internet Technol.*, vol. 23, no. 3, Aug. 2023. [Online]. Available: https://doi.org/10.1145/3511899

[5] Z. Du, Y. Li, Y. Fu, and X. Zheng, "Blockchain-based access control architecture for multi-domain environments," *Pervasive and Mobile Computing*, vol. 98, p. 101878, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S157411922400004X

[6] D. Han, Y. Zhu, D. Li, W. Liang, A. Souri, and K.-C. Li, "A blockchain-based auditable access control system for private data in service-centric iot environments," *IEEE Transactions on Industrial Informatics*, vol. 18, no. 5, pp. 3530–3540, 2022.

[7] T. Hu, S. Yang, Y. Wang, G. Li, Y. Wang, G. Wang, and M. Yin, "N-accesses: A blockchain-based access control framework for secure iot data management," *Sensors*, vol. 23, no. 20, 2023. [Online]. Available: https://www.mdpi.com/1424-8220/23/20/8535

[8] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-first software: You own your data, in spite of the cloud," in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*. ACM, 2019, pp. 154–178.

[9] F. Jacob and H. Hartenstein, "To the best of knowledge and belief: On eventually consistent access control," in *Proceedings of the 15th ACM Conference on Data and Application Security and Privacy*, 2024.

[10] A. R. Nasab, M. Shahin, S. A. H. Raviz, P. Liang, A. Mashmool, and V. Lenarduzzi, "An empirical study of security practices for microservices systems," *Journal of Systems and Software*, vol. 198, p. 111563, 2023.

[11] S. Qi, Y. Zheng, M. Li, Y. Liu, and J. Qiu, "Scalable industry data access control in rfid-enabled supply chain," *IEEE/ACM Transactions on Networking*, vol. 24, no. 6, pp. 3551–3564, 2016.

[12] S. Rouhani, R. Belchior, R. S. Cruz, and R. Deters, "Distributed attribute-based access control system using permissioned blockchain," *World Wide Web*, vol. 24, no. 5, pp. 1617–1644, 2021.

[13] A. Singh, G. Kumar, R. Saha, M. Conti, M. Alazab, and R. Thomas, "A survey and taxonomy of consensus protocols for blockchains," *Journal of Systems Architecture*, vol. 127, p. 102503, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1383762122000777

[14] Y. Zhao, B. Tian, Y. Niu, H. Zhang, Z. Yi, and R. Zeng, "A security management and control solution of smart park based on sensor networks," *Sensors*, vol. 21, no. 20, 2021. [Online]. Available: https://www.mdpi.com/1424-8220/21/20/6815

[15] Y. Zhang, S. Kasahara, Y. Shen, X. Jiang, and J. Wan, "Smart contract-based access control for the internet of things," *IEEE Internet of Things Journal*, vol. 6, no. 2, pp. 1594–1605, 2019.

[16] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-free replicated data types," in *Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*, ser. Lecture Notes in Computer Science, vol. 6976. Springer, 2011, pp. 386–400.

[17] V. B. F. Gomes, M. Kleppmann, D. Mulligan, and A. R. Beresford, "Verifying strong eventual consistency in distributed systems," *Proceedings of the ACM on Programming Languages*, vol. 1, no. OOPSLA, 2017.

[18] G. Yu, X. Zha, X. Wang, W. Ni, K. Yu, P. Yu, J. A. Zhang, R. P. Liu, and Y. J. Guo, "Enabling attribute revocation for fine-grained access control in blockchain-iot systems," *IEEE Transactions on Engineering Management*, vol. 67, no. 4, pp. 1213–1230, 2020.

[19] D. Han, N. Pan, and K.-C. Li, "A traceable and revocable ciphertext-policy attribute-based encryption scheme based on privacy protection," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 1, pp. 316–327, 2022.

[20] M. Xiao, Q. Huang, Y. Miao, S. Li, and W. Susilo, "Blockchain based multi-authority fine-grained access control system with flexible revocation," *IEEE Transactions on Services Computing*, vol. 15, no. 6, pp. 3143–3155, 2022.

[21] Y. Miao, R. H. Deng, X. Liu, K.-K. R. Choo, H. Wu, and H. Li, "Multi-authority attribute-based keyword search over encrypted cloud data," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 4, pp. 1667–1680, 2021.

[22] K. Yue, Y. Zhang, Y. Chen, Y. Li, L. Zhao, C. Rong, and L. Chen, "A survey of decentralizing applications via blockchain: The 5g and beyond perspective," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2191–2217, 2021.

[23] D. Di Francesco Maesa, P. Mori, and L. Ricci, "A blockchain based approach for the definition of auditable access control systems," *Computers & Security*, vol. 84, pp. 93–119, 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167404818309398

[24] H. Rathore, A. Mohamed, and M. Guizani, "A survey of blockchain enabled cyber-physical systems," *Sensors*, vol. 20, no. 1, 2020. [Online]. Available: https://www.mdpi.com/1424-8220/20/1/282

[25] M. M. Merlec, M. M. Islam, Y. K. Lee, and H. P. In, "A consortium blockchain-based secure and trusted electronic portfolio management scheme," *Sensors*, vol. 22, no. 3, 2022. [Online]. Available: https://www.mdpi.com/1424-8220/22/3/1271

[26] A. Birgisson, J. G. Politz, U. Erlingsson, A. Taly, M. Vrable, and M. Lentczner, "Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud," in *Proceedings of the 2014 Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2014.

[27] B. Zelenka, P. Krüger *et al.*, "UCAN specification v0.10.0," https://github.com/ucan-wg/spec, 2022, accessed: 2025.

[28] M. P. Andersen, S. Kumar, M. AbdelBaky, G. Fierro, J. Kolb, H.-S. Kim, D. E. Culler, and R. A. Popa, "WAVE: A decentralized authorization framework with transitive delegation," in *Proceedings of the 28th USENIX Security Symposium*. USENIX Association, 2019, pp. 1375–1392.

[29] S. M. H. Bamakan, A. Motavali, and A. Bondarti, "A survey of blockchain consensus algorithms performance evaluation criteria," *Expert Systems with Applications*, vol. 154, p. 113385, 2020.

[30] C. Fan, S. Ghaemi, H. Khazaei, and P. Musilek, "Performance evaluation of blockchain systems: A systematic survey," *IEEE Access*, vol. 8, pp. 126 927–126 950, 2020.

[31] M. Touloupou, M. Themistocleous, E. Iosif, and K. Christodoulou, "A systematic literature review toward a blockchain benchmarking framework," *IEEE Access*, vol. 10, pp. 70 630–70 644, 2022.

[32] Y. Xiao, N. Zhang, W. Lou, and Y. T. Hou, "A survey of distributed consensus protocols for blockchain networks," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 2, pp. 1432–1465, 2020.

[33] W. Wang, D. Hoang, P. Hu, Z. Xiong, D. Niyato, P. Wang, Y. Wen, and D. I. Kim, "A survey on consensus mechanisms and mining strategy management in blockchain networks," *IEEE Access*, vol. 7, pp. 22 328–22 370, 2019.

[34] M. Ferdous, M. Chowdhury, and M. A. Hoque, "A survey of consensus algorithms in public blockchain systems for cryptocurrencies," *Journal of Network and Computer Applications*, vol. 182, p. 103035, 2021.

[35] C. T. Nguyen, D. T. Hoang, D. N. Nguyen, D. Niyato, H. T. Nguyen, and E. Dutkiewicz, "Proof-of-stake consensus mechanisms for future blockchain networks: Fundamentals, applications and opportunities," *IEEE Access*, vol. 7, pp. 85 727–85 745, 2019.

[36] W. Li, C. Feng, L. Zhang, H. Xu, B. Cao, and M. Imran, "A scalable multi-layer PBFT consensus for blockchain," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 5, pp. 1146–1160, 2021.

[37] G. Couprie *et al.*, "Biscuit: Bearer tokens with offline attenuation and decentralized verification," https://www.biscuitsec.org/, 2023, accessed: 2025.

[38] A. Meyer and S. Gwilym, "Willow protocol specification," https://willowprotocol.org/, 2024, includes Meadowcap capability system. Accessed: 2025.

[39] R. Pang, R. Caceres, M. Burrows, Z. Chen, P. Dave, N. Gerber, A. Giber, A. Narayan, and F. Boemer, "Zanzibar: Google's consistent, global authorization system," in *Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC)*. USENIX Association, 2019, pp. 33–46.

[40] Auth0/Okta, "OpenFGA: An open-source authorization solution," https://openfga.dev/, 2023, cNCF Sandbox project. Accessed: 2025.

[41] J. W. Cutler, C. Disselkoen, A. Eline, S. He, K. Headley, M. Hicks, K. Hietala, E. Ioannidis, J. Kastner, A. Mamat, D. McAdams, M. McCutchen, N. Rungta, E. Torlak, and A. Wells, "Cedar: A new language for expressive, fast, analyzable, and secure authorization," *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, 2024.

[42] The Matrix.org Foundation, "Matrix specification: State resolution v2," https://spec.matrix.org/v1.8/rooms/v2/, 2023, accessed: 2025.

[43] X. Yao, H. Kong, H. Liu, T. Qiu, and H. Ning, "An attribute credential based public key scheme for fog computing in digital manufacturing," *IEEE Transactions on Industrial Informatics*, vol. 15, no. 4, pp. 2297–2307, 2019.

[44] B. H. Kim and D. Lie, "Caelus: Verifying the consistency of cloud services with battery-powered devices," in *2015 IEEE Symposium on Security and Privacy*, 2015, pp. 880–896.

[45] T. Bui and T. Aura, "Application of public ledgers to revocation in distributed access control," in *Information and Communications Security*, D. Naccache, S. Xu, S. Qing, P. Samarati, G. Blanc, R. Lu, Z. Zhang, and A. Meddahi, Eds. Cham: Springer International Publishing, 2018, pp. 781–792.

[46] A. A. Monrat, O. Schelén, and K. Andersson, "A survey of blockchain from the perspectives of applications, challenges, and opportunities," *IEEE Access*, vol. 7, pp. 117 134–117 151, 2019.

[47] N. Deepa, Q.-V. Pham, D. C. Nguyen, S. Bhattacharya, B. Prabadevi, T. R. Gadekallu, P. K. R. Maddikunta, F. Fang, and P. N. Pathirana, "A survey on blockchain for big data: Approaches, opportunities, and future directions," *Future Generation Computer Systems*, vol. 131, pp. 209–226, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167739X22000243

[48] M. Kohlweiss and I. Miers, "Accountable metadata-hiding escrow: A group signature case study," *Proceedings on Privacy Enhancing Technologies*, 2015.

[49] J. Bernal Bernabe, J. L. Canovas, J. L. Hernandez-Ramos, R. Torres Moreno, and A. Skarmeta, "Privacy-preserving solutions for blockchain: Review and challenges," *IEEE Access*, vol. 7, pp. 164 908–164 940, 2019.

[50] M. Ma, G. Shi, and F. Li, "Privacy-oriented blockchain-based distributed key management architecture for hierarchical access control in the iot scenario," *IEEE Access*, vol. 7, pp. 34 045–34 059, 2019.

[51] Y. Zhang, R. H. Deng, S. Xu, J. Sun, Q. Li, and D. Zheng, "Attribute-based encryption for cloud computing access control: A survey," *ACM Computing Surveys (CSUR)*, vol. 53, no. 4, pp. 1–41, 2020.

Aditya Sissodiya received his B.Sc. (Hons.) degree in Computer Science from Doon University, and his M.Sc. degree in Computer Science from Christ University, India. He is pursuing a Ph.D. in Cyber-Physical Systems at Luleå University of Technology, Sweden. Before beginning his doctoral studies, he worked as a software engineer for 4 years in industry, contributing to supply chain optimization and healthcare technologies. His research interests lie in access control and secure collaboration mechanisms, with a recent focus on technologies supporting the circular economy.

Eric Chiquito received his B.S. degree in nanotechnology engineering from The Instituto Tecnológico y de Estudios Superiores de Occidente (ITESO), Mexico in. He concluded his PhD degree in cyber-physical systems at Luleå University of Technology, Sweden on 2024, and currently works as a postdoctoral researcher at LTU. Prior to this, he worked as a data scientist in the largest hotel group in Mexico, Grupo Posadas. His research interests include decentralized systems, data sharing, circular economy, and matchmaking mechanisms.

Johan Kristiansson received his Ph.D. in multimedia technology from Luleå University of Technology, where he is currently employed as a Senior Lecturer in Cyber-Physical Systems. He has extensive industry experience, including over a decade at Ericsson, where he worked as a Principal Researcher specialising in container orchestration. As an early adopter of Docker, he explored microservices architectures and container security, and developed an in-house orchestration platform at Ericsson in 2014. He later served as CTO at RockSigma AB, where he built a Kubernetes-based seismic processing engine. He is also the lead developer of ColonyOS, a meta-OS designed to create computing continuums. Dr. Kristiansson has authored over 25 scientific publications and holds 23 patents in distributed computing, cloud computing, and container orchestration.

Ulf Bodin is a professor at the Luleå University of Technology, where he is conducting research on the industrial IoT, distributed system of systems, computer communications, distributed ledgers and applied machine learning. He has a PhD in computer networking from Luleå University of Technology and more than 15 years of experience in academia and the software industry, including standardization in ETSI and other organizations.